

گزارش پروژه برنامه نویسی پیشرفته - فاز ۱

به نام خدا

استاد مدرس: مرتضی یوسف صنعت | دانشجو: محمدامین ترکمان (۴۰۴۱۲۳۵۸۰۱۲)

فهرست مطالب

• مقدمه ۲	• کلاس Board ۳
• کلاس ها ۲	• کلاس Card ۳
• کلاس Heroes ۲	• کلاس CardResolver ۴
• کلاس Dcacula ۲	• کلاس Player ۴
• کلاس SherlockHolmes ۲	• کلاس Ftxui_Front ۴
• کلاس Sisters ۲	• کلاس Controller ۵
• کلاس Dr_Watsone ۳	• مراجع ۵
• کلاس Space ۳	

مقدمه

در این فاز از پروژه، ساختار بک اند (بخش پشتی) و منطق اصلی بازی پیاده سازی شده است. به عنوان هسته مرکزی سیستم، این بخش مسئولیت مدیریت وضعیت بازی، قوانین، کارت ها، بازیکنان و تعاملات منطقی بین اجزای مختلف را بر عهده دارد و پایه ای پایدار و مهندسی شده را برای لایه های نمایشی فراهم می کند.

کلاس ها

کلاس Heroes

این کلاس به عنوان پایه و ساختار اصلی برای تعریف و مدیریت ویژگی های قهرمانان و شخصیت های مختلف در بازی عمل می کند و مشخصات کلیدی آن ها را نگه می دارد.

متدهای کلیدی:

- **DrawnCard():** شبیه سازی کشیدن یک کارت از دسته کارت ها (Deck). ابتدا بررسی می کند که آیا دسته کارت خالی است یا خیر؛ اگر خالی باشد ۰ برمی گرداند. در غیر این صورت، کارت بالایی را برداشته، به دست بازیکن اضافه می کند، آن را از دسته کارت حذف کرده و ۱ برمی گرداند.
- **Damage(int amount):** به قهرمان آسیب وارد می کند. اگر میزان سلامتی به صفر یا کمتر برسد، وضعیت زنده بودن قهرمان را برابر نادرست (false) قرار داده، قهرمان را از خانه فعلی روی تخته حذف می کند و اشاره گر خانه را پاک می کند؛ در غیر این صورت، مقدار آسیب را از سلامتی کم می کند.

- **discard_hand():** به صورت تصادفی یک کارت را از دست بازیکن انتخاب کرده، مقدار تقویت (boost) آن را دریافت می‌کند، کارت را به دسته کارت‌های سوخته (discard pile) منتقل کرده و از دست حذف می‌کند.
- **Discard_Card(Card *card):** یک اشاره‌گر کارت خاص را در دست جستجو می‌کند. اگر پیدا شود، کارت را به دسته کارت‌های سوخته منتقل کرده و از دست پاک می‌کند؛ در غیر این صورت، یک خطای زمان اجرا (runtime error) پرتاب می‌کند.

کلاس Dacula

کلاس مرتبط با شخصیت دراکولا (Dracula) که منطق، قابلیت‌ها و رفتارهای خاص این شخصیت منحصر به فرد یا شرور را در طول بازی پیاده‌سازی می‌کند.

کلاس SherlockHolmes

این کلاس مسئول پیاده‌سازی ویژگی‌ها، مهارت‌ها و رفتارهای شرلوک هولمز به عنوان یکی از شخصیت‌های اصلی و استراتژیک در بازی است.

متدها و مقداردهی اولیه کلیدی:

- **سازنده SherlockHolmes:** شرلوک هولمز را با آمارهای اصلی (نام، نوع حمله تن‌به‌تن، ۱۶ سلامتی، ۲ حرکت) مقداردهی اولیه کرده و دسته کارت‌های او را با کارت‌های اختصاصی شخصیت و عمومی (مانند Administer Aid، و غیره) پر می‌کند. سپس دسته کارت را به صورت تصادفی بر زده و دست اولیه ۵ کارتی را Counterpunch، Feint می‌کشد.

کلاس Sisters

کلاس مربوط به خواهران یا شخصیت‌های فرعی مرتبط، که تعاملات گروهی و قوانین خاص مربوط به حضور آن‌ها در روند بازی را مدیریت می‌کند.

کلاس Dr_Watsone

این کلاس قابلیت‌ها و منطق مرتبط با دکتر واتسون را تعریف و کنترل کرده و نقش حمایتی یا ابزاری او را در بازی هدایت می‌کند.

کلاس Space

این کلاس فضاها، خانه‌ها یا مناطق مختلف تخته بازی را که مهره‌ها و کارت‌ها روی آن‌ها قرار می‌گیرند، مدل‌سازی و مدیریت می‌کند.

متدهای کلیدی:

- **سازنده Space:** یک خانه خاص روی تخته را با تنظیم شماره شناسه منحصر به فرد، مناطق مرتبط (بردار فضاها) و خانه‌های مجاور آن مقداردهی اولیه می‌کند.
- **get_zone():** برداری شامل اشاره‌گرها به تمام اشیاء فضایی که دارای منطقه یکسانی با این خانه هستند را برمی‌گرداند که برای ردیابی حملات از راه دور و اثرات منطقه‌ای ضروری است.

- **get_neighbor():** برداری از خانه‌های مجاور متصل به این خانه را برمی‌گرداند که عمدتاً برای حرکت و منطق موقعیتیابی استفاده می‌شود.
- **get_hero() & set_hero(Heroes *hero):** اشاره‌گر قهرمانی که در حال حاضر این خانه را اشغال کرده است بازیابی می‌کند یا قهرمانی را در طول اقدامات حرکتی روی این خانه قرار می‌دهد/حذف می‌کند.
- **get_number():** شناسه عددی منحصر به فرد فضا را برمی‌گرداند.

کلاس Board

تخته اصلی بازی را نمایش می‌دهد و مسئولیت مدیریت چیدمان خانه‌ها، موقعیت بازیکنان و وضعیت کلی ساختار تخته را بر عهده دارد.

متدها و پیاده‌سازی کلیدی:

- **سازنده Board:** تخته بازی را با تغییر اندازه بردار فضاها به ۳۲ خانه و تعریف گراف اتصال دقیق، از جمله ارتباطات منطقه‌ای و همسایگان فوری برای هر فضای فردی، مقداردهی اولیه می‌کند.
- **is_Adjacent(Space *s1, Space *s2):** با تکرار در لیست همسایگان فضای اول، بررسی می‌کند که آیا دو اشاره‌گر فضای داده شده مستقیماً با یکدیگر مجاور هستند یا خیر، و اگر ارتباطی وجود داشته باشد، مقدار درست را برمی‌گرداند (true).
- **get_spaces():** ارجاعی به کل مجموعه فضاها را تشکیل‌دهنده ساختار تخته را برمی‌گرداند.
- **SwapHeroes(Heroes *hero1, Heroes *hero2):** با جابجایی ایمن موقعیت‌ها و اشاره‌گرهای فضای تخته مرتبط دو قهرمان، جابجایی تاکتیکی را مدیریت می‌کند و اگر هر یک از قهرمانان نامعتبر یا مستقر نشده باشند، خطای زمان اجرا پرتاب می‌کند.

کلاس Card

کلاس پایه برای تعریف کارت‌های بازی، نگهداری ویژگی‌هایی مانند نوع کارت، اثرات و اطلاعات مربوط به هر کارت.

کلاس CardResolver

این کلاس مسئول تفسیر، پردازش و اجرای اثرات کارت پس از بازی شدن توسط بازیکنان است و منطق قوانین کارت را مدیریت می‌کند.

متدها و تفاسیر کلیدی:

- **execute():** این متد حجیم و کلیدی از یک ساختار switch-case بزرگ روی نوع کارت (CardType) استفاده می‌کند تا بررسی کند چه کارتی بازی شده است. سپس بسته به اینکه کارت متعلق به کدام بازیکن یا قهرمان است (مانند دراکولا، شرلوک هولمز، خواهران یا دکتر واتسون)، اثرات خاص آن کارت را اعمال می‌کند؛ اثراتی مانند اصلاح مقادیر حمله و دفاع (Attack_Value و Defence_Value)، حرکت قهرمانان روی تخته، اعمال یا کاهش آسیب و سلامتی، کشیدن کارت از دست (DrawnCard) یا غیرفعال کردن قابلیت‌های حریف.
- **TryDisableCard():** این متد کمکی برای مدیریت کارت‌های فینت و خنثی‌سازی (مانند Feint) طراحی شده است. ابتدا بررسی می‌کند که آیا کارت معتبری ارسال شده است یا خیر. سپس بررسی می‌کند که آیا مالک کارت شرلوک

هولمز یا دکتر واتسون است و شرلوک زنده است، از کارت در برابر غیرفعال شدن محافظت می‌کند؛ در غیر این صورت، با فراخوانی `set_ApplyEffects(false)`، اثرات کارت حریف را خنثی و بی‌اعتبار می‌کند.

کلاس Player

اطلاعات مربوط به بازیکنان، امتیازات، کارت‌های در دست و وضعیت هر کاربر در طول بازی را مدیریت و نگهداری می‌کند.

کلاس Ftxui_Front

این کلاس تعامل با لایه ارائه مبتنی بر FTXUI را مدیریت می‌کند و یک رابط کاربری متنی و جذاب را در محیط ترمینال ارائه می‌دهد.

متدها و تفاسیر کلیدی:

- **Menu_():** این متد یک منوی تعاملی تمام صفحه برای صفحه اصلی بازی (شامل گزینه‌های بازی، دستورالعمل‌ها و خروج) ایجاد می‌کند. از کتابخانه FTXUI برای پردازش رویدادهای صفحه کلید (مانند کلید Enter) استفاده می‌کند و مقدار شمارش شده `e_Menu` مناسب را بر اساس انتخاب کاربر برمی‌گرداند.
- **Players_Info_List(Player *p1, Player *p2):** بازیکنان (نام و سن) به همراه یک دکمه تایید پیاده‌سازی می‌کند. پس از تایید کاربر، اطلاعات وارد شده را مستقیماً روی اشیاء بازیکن تنظیم و ذخیره می‌کند.
- **Det_characters(Player *p1, Player *p2):** سن بازیکنان را مقایسه می‌کند تا بازیکن جوان‌تر در انتخاب قهرمان اولویت داشته باشد، سپس منویی برای انتخاب شخصیت (مانند دراکولا یا شرلوک هولمز) نمایش می‌دهد و انتخاب نهایی را برمی‌گرداند.
- **catch_place(Player *p1, Player *p2, Board *board):** طریق یک منوی گرافیکی دریافت می‌کند و با در نظر گرفتن سن بازیکنان، موقعیت‌های تخته (شماره‌های ۳ و ۲۳) را به قهرمانان اختصاص می‌دهد.
- **Graph_Box(vector<Space> &spaces):** نقشه کامل و گراف تخته را در یک بوم (Canvas) رندر می‌کند. این متد خطوط اتصال بین فضاها و وضعیت قرارگیری قهرمانان روی هر خانه (همراه با رنگ‌ها و اطلاعات مختصر) را به صورت بصری و پویا در ترمینال رندر می‌کند.
- **choose_comrad_place(Player *p1, Player *p2, Board *board):** بازیکن (سه خواهر برای دراکولا و دکتر واتسون برای شرلوک) را در مناطق مجاز روی تخته مدیریت کرده و پس از تایید، آن‌ها را در خانه انتخابی قرار می‌دهد.
- **Attakcer_Heroes_Menu(Player *player, Board *board, Heroes *&hero):** لیست مبارزان زنده متعلق به بازیکن (شامل قهرمان اصلی و یاران) را استخراج کرده و منویی برای انتخاب مهاجم فعلی در طول نوبت فراهم می‌کند.
- **Defender_Heroes_Menu(Player *player, Board *board, Heroes *&defender, Heroes *&Attacker):** مدافعان حریف واجد شرایط را بر اساس نوع حمله (تن‌به‌تن یا از راه دور) و با بررسی مسیرها و مجاورت خانه‌ها فیلتر می‌کند، سپس منوی انتخاب مدافع را برای کاربر رندر می‌کند.

- **Attacker_selected_card(Heroes *Attacker, Heroes *Defender, Player *p1, Player *p2, Board *board, Card *&Attacker_Card):** کارت‌های موجود در دست مهاجم را که با نوع حمله سازگار هستند فیلتر می‌کند، منوی انتخاب کارت حمله را نمایش می‌دهد و کارت انتخاب شده را نهایی می‌کند.
- **Defender_selected_card(Heroes *Attacker, Heroes *Defender, Player *p1, Player *p2, Board *board, Card *&Defender_Card):** کارت‌های دفاع مناسب را از دست مدافع (یا یاران مرتبط) استخراج می‌کند، گزینه «بدون دفاع» را فراهم می‌کند و کارت دفاع انتخاب شده را اعمال می‌کند.
- **MoveHero(Heroes *hero, Board *board, int max_distance):** از الگوریتم‌های جستجوی گراف (BFS) برای یافتن تمام خانه‌های قابل دسترس تا یک فاصله مشخص از مکان فعلی قهرمان استفاده می‌کند و یک منوی انتخاب مقصد را به کاربر ارائه می‌دهد.
- **SelectHero(Board *board):** تمام مبارزان زنده موجود روی تخته بازی را اسکن و جمع‌آوری کرده و یک منوی تعاملی برای انتخاب یکی از آن‌ها برای عملیات مختلف فراهم می‌کند.
- **PlaceHeroIntroduction = True**
- **PlaceHeroAdjacent(Heroes *hero, Heroes *target, Board *board):** اطراف یک هدف خاص پیدا می‌کند و به کاربر اجازه می‌دهد قهرمان مورد نظر را در آن موقعیت‌های اطراف قرار دهد.
- **DiscardCards(Heroes *dracula):** به بازیکن اجازه می‌دهد به طور تعاملی کدهای یا کارت‌های خاصی را از دست یک قهرمان (مانند دراکولا) انتخاب و حذف کند تا فرآیند دور انداختن کارت تکمیل شود.
- **Event_Selected_Card(...):** کارت‌های نوع رویداد را از دست بازیکن فیلتر می‌کند، رابط انتخاب کارت رویداد را نمایش می‌دهد و کارت را برای اجرای اثرات خاص آماده می‌کند.

کلاس Controller

کنترلر مرکزی بازی که جریان اجرای برنامه را هدایت می‌کند، ورودی‌ها را دریافت می‌کند، وضعیت‌ها را به‌روزرسانی می‌کند و بین اجزای مختلف یک‌اند و فرانت‌اند هماهنگی ایجاد می‌کند.

متدها و تفاسیر کلیدی:

- **set_players_character(int choise):** نقش شخصیت‌ها (دراکولا، شرلوک هولمز و همراهانشان) را بر اساس انتخاب کاربر و سن بازیکنان به بازیکنان اختصاص می‌دهد و تعیین می‌کند چه کسی کدام جناح را بازی کند.
- **get_younger_player() & get_older_player():** سنین هر دو بازیکن (p1 و p2) را مقایسه می‌کند تا اشاره‌گری به بازیکن جوان‌تر (اولویت کاربر) یا بازیکن مسن‌تر را شناسایی کرده و برگرداند.
- **Initial_turn():** نوبت اولیه بازی را با تعیین بازیکن جوان‌تر به عنوان بازیکن نوبت فعال و دیگری به عنوان حریف تنظیم می‌کند.
- **chane_turn, run():** نوبت فعال را تغییر می‌دهد و حلقه مرکزی بازی و موتور اجرایی را اجرا می‌کند.

مراجع

- C++ How to program Book
- <https://github.com/ArthurSonzogni/FTXUI.git>
- <https://github.com/>
- <https://claude.ai/>